
TUGAS AKHIR TEKNIK KOMPILASI
REPRESENTASI 4 TAHAP KOMPILASI
KONSTRUKSI: WHILE LOOP

Disusun oleh:

Nama : MELSA SENTIA ASTA

NIM : 231011402659

Kelas : 06 TPLE 002

Mata Kuliah : Teknik Kompilasi

BAB I

PENDAHULUAN

1.1 Latar Belakang

Compiler adalah program yang menerjemahkan source code (bahasa tingkat tinggi) menjadi bahasa mesin yang dapat dimengerti oleh komputer. Proses kompilasi terdiri dari beberapa tahapan, yaitu:

1. **Lexical Analysis** - Mengubah teks menjadi token
2. **Syntax Analysis** - Memeriksa struktur berdasarkan grammar
3. **Semantic Analysis** - Memeriksa makna/logika program
4. **Code Generation** - Menghasilkan kode antara atau kode mesin

Tugas ini bertujuan untuk mensimulasikan keempat tahapan tersebut pada konstruksi **perulangan while**.

1.2 Rumusan Masalah

1. Bagaimana implementasi 4 tahapan kompilasi (Lexical Analysis, Syntax Analysis, Semantic Analysis, dan Three-Address Code Generation) pada konstruksi perulangan while loop?
2. Bagaimana program menangani kesalahan (error handling) yang mungkin terjadi selama proses kompilasi?
3. Bagaimana implementasi program mengkaitkan konsep-konsep teori kompilasi ke dalam praktik?

1.3 Tujuan

1. Memahami alur kerja compiler dari source code hingga kode antara (TAC)
2. Mengimplementasikan konsep BNF Grammar dalam parsing
3. Membangun Abstract Syntax Tree (AST) dari source code
4. Menghasilkan Three-Address Code yang siap untuk optimasi

1.4 Pemilihan Konstruksi

1. While loop adalah konstruksi fundamental dalam pemrograman
2. Banyak digunakan dalam berbagai bahasa pemrograman
3. Memiliki struktur yang jelas untuk dikompilasi (kondisi + body)
4. Menunjukkan konsep kontrol aliran (branching dan looping)

BAB II PEMBAHASAN

2.1 Definisi BNF Grammar

```
1 <program>      ::= <while_stmt>
2
3 <while_stmt>   ::= "while" "(" <condition> ")" "{" <statement> "}"
4
5 <condition>    ::= <identifier> <rel_op> <number>
6
7 <statement>    ::= <assignment>
8
9 <assignment>  ::= <identifier> "=" <expression> ";"
10
11 <expression>  ::= <identifier> "+" <number>
12                | <identifier> "-" <number>
13                | <number>
14                | <identifier>
15
16 <rel_op>      ::= "<" | ">" | "<=" | ">=" | "==" | "!="
17
18 <identifier>  ::= letter (letter | digit)*
19
20 <number>      ::= digit+
```

Penjelasan Grammar:

Simbol	Arti
<program>	Program utama yang terdiri dari satu while statement
<while_stmt>	Struktur while: while (kondisi) { pernyataan }
<condition>	Kondisi perulangan: variabel operator angka
<statement>	Pernyataan di dalam body: assignment
<assignment>	Assignment: variabel = ekspresi ;
<expression>	Ekspresi: operasi penjumlahan/pengurangan atau nilai tunggal
<rel_op>	Operator relasional: <, >, <=, >=, ==, !=
<identifier>	Nama variabel (huruf/garis bawah diikuti huruf/angka)
<number>	Angka bulat (digit 0-9)

2.2 Implementasi Program

Program diimplementasikan menggunakan **Python 3** dengan pendekatan **Object-Oriented**.

2.2.1 Lexical Analysis

Fungsi: Memecah source code menjadi token-token bermakna.

Cara kerja:

1. Mendefinisikan pola token dalam bentuk Regular Expression
2. Menggunakan `re.finditer()` untuk mencocokkan pola
3. Setiap token disimpan dalam bentuk tuple (jenis, nilai)
4. Spasi dan newline diabaikan (SKIP)

Kode:

```
1 def lexical_analysis(self):
2     """Tahap 1: Memecah source code menjadi token"""
3     token_spec = [
4         ('WHILE', r'while'),
5         ('NUMBER', r'\d+'),
6         ('ID', r'[A-Za-z_]\w*'),
7         ('RELOP', r'<|>|=|!=|<|>'),
8         ('ASSIGN', r'='),
9         ('PLUS', r'\+'),
10        ('MINUS', r'-'),
11        ('LPAREN', r'('),
12        ('RPAREN', r')'),
13        ('LBRACE', r'\{'),
14        ('RBRACE', r'\}'),
15        ('SEMICOLON', r';'),
16        ('SKIP', r'[\t\n]+'),
17    ]
```

Output:

```
--- Generasi Three-Address Code (TAC) ---
--- Hasil Analisis Leksikal (Tokens) ---
('WHILE', 'while')
('LPAREN', '(')
('ID', 'x')
('RELOP', '<')
('NUMBER', '10')
('RPAREN', ')')
('LBRACE', '{')
('ID', 'x')
('ASSIGN', '=')
('ID', 'x')
('PLUS', '+')
('NUMBER', '1')
('SEMICOLON', ';')
('RBRACE', '}')
```

2.2.2 Syntax Analysis

Fungsi: Memeriksa urutan token sesuai BNF grammar dan membangun AST.

Cara kerja:

1. Parser membaca token satu per satu (top-down parsing)
2. Menggunakan fungsi eat() untuk mengonsumsi token yang diharapkan
3. Jika token tidak sesuai, menghasilkan SyntaxError
4. Membangun AST dalam bentuk dictionary Python

Kode:

```
1  return {
2      'type': 'While',
3      'condition': {
4          'left': left,
5          'op': op,
6          'right': right
7      },
8      'body': {
9          'target': target,
10         'left': left_exp,
11         'operator': operator,
12         'value': value
13     }
14 }
```

Output:

```
172
173 # --- Contoh Penggunaan ---
174 source = "while (x < 10 { x = x + 1; }"
175 compiler = WhileCompiler(source)
176
```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS

File "d:\Tugas_Kompilasi_While\CompilerWhile.py", line 112, in syntax_analysis
raise SyntaxError(f"Syntax Error: {str(e)}")
SyntaxError: Syntax Error: Expected RPAREN, found LBRACE
PS D:\Tugas_Kompilasi_While> █

Error Handling:

- Jika token tidak sesuai urutan maka SyntaxError
- Contoh diatas: while (x < 10 { x = x + 1; } (kurang kurung tutup)

2.2.3 Semantic Analysis

Fungsi: Memeriksa makna dan konsistensi logika program.

Aturan yang diperiksa:

1. Variabel di kondisi harus konsisten dengan variabel di body
2. Variabel target assignment harus sama dengan left operand
3. Mencegah infinite loop (variabel kondisi harus berubah di body)
- 4.

Kode:

```
114 def semantic_analysis(self, ast):
115     """Tahap 3: Memeriksa konsistensi semantik"""
116     cond = ast['condition']
117     body = ast['body']
118
119     # Cek konsistensi variabel
120     if cond['left'] != body['target']:
121         raise Exception(
122             f"Semantic Error: Condition uses '{cond['left']}' "
123             f"but body modifies '{body['target']}'"
124         )
125
126     if body['left'] != body['target']:
127         raise Exception(
128             f"Semantic Error: Assignment target must match left operand"
129         )
130
131     return True
```

Output dan Contoh Error:

```
173 # --- Contoh Penggunaan ---
174 source = "while (x < 10) { y = y + 1; }"
175 compiler = WhileCompiler(source)
176
PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS

--- Hasil Syntax Analysis (AST) ---
{'type': 'While', 'condition': {'left': 'x', 'op': '<', 'right': '10'}, 'body': {'target': 'y', 'left': 'y', 'operator': '+', 'value': '1'}}
Traceback (most recent call last):
  File "d:\Tugas_Kompilasi_While\CompilerWhile.py", line 183, in <module>
    print(compiler.generate_tac())
    ~~~~~^
  File "d:\Tugas_Kompilasi_While\CompilerWhile.py", line 149, in generate_tac
    self.semantic_analysis(ast)
  File "d:\Tugas_Kompilasi_While\CompilerWhile.py", line 121, in semantic_analysis
    raise Exception(
Exception: Semantic Error: Condition uses 'x' but body modifies 'y'
PS D:\Tugas_Kompilasi_While>
```

```
while (x < 10) {
    y = y + 1; } // ERROR: x tidak pernah berubah!
Semantic Error: Condition uses 'x' but body modifies 'y'
```

2.2.4 Three-Address Code Generation

Fungsi: Mengubah AST menjadi Three-Address Code (TAC).

Karakteristik TAC:

1. Setiap instruksi maksimal 3 operand
2. Menggunakan label (L1, L2) untuk kontrol aliran
3. Menggunakan variabel temporary (t1, t2) untuk hasil antara

Kode dan Output TAC:

```
157     loop_start = self.new_label()
158     loop_end = self.new_label()
159     temp = self.new_temp()
160
161     tac = []
162     tac.append(f"{loop_start}:")
163     tac.append(f"if {cond['left']} {cond['op']} {cond['right']} goto {loop_start}")
164     tac.append(f"goto {loop_end}")
165     tac.append(f"{temp} = {body['left']} {body['operator']} {body['value']}")
166     tac.append(f"{body['target']} = {temp}")
167     tac.append(f"goto {loop_start}")
168     tac.append(f"{loop_end}:")
169     return "\n".join(tac)
170 # --- Contoh Penggunaan ---
171 source = "while (x < 10) { x = x + 1; }"
172 compiler = WhileCompiler(source)
173
```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
--- Semantic Analysis ---
✓ Semantic OK
L1:
if x < 10 goto L1
goto L2
t1 = x + 1
x = t1
goto L1
L2:
```

2.3. Contoh Output

2.3.1 Program Dijalankan

Source Code:

```
170 # --- Contoh Penggunaan ---
171 source = "while (x < 10) { x = x + 1; }"
```

Output Lengkap:

```
--- Generasi Three-Address Code (TAC) ---
--- Hasil Analisis Leksikal (Tokens) ---
('WHILE', 'while')
('LPAREN', '(')
('ID', 'x')
('RELOP', '<')
('NUMBER', '10')
('RPAREN', ')')
('LBRACE', '{')
('ID', 'x')
('ASSIGN', '=')
('ID', 'x')
('PLUS', '+')
('NUMBER', '1')
('SEMICOLON', ';')
('RBRACE', '}')

--- Hasil Syntax Analysis (AST) ---
{'type': 'While', 'condition': {'left': 'x', 'op': '<', 'right': '10'}, 'body': {'target': 'x', 'left': 'x', 'operator': '+', 'value': '1'}}

--- Semantic Analysis ---
✓ Semantic OK
L1:
if x < 10 goto L1
goto L2
t1 = x + 1
x = t1
goto L1
L2:
=====
PS D:\Tugas_Kompilasi_While>
```

2.4 Kaitan Dengan Teknik Kompilasi

Tahap	Konsep Teori	Implementasi
Lexical	Regular Expression & Finite Automata	Menggunakan re.finditer() dengan pola regex
Syntax	Context-Free Grammar (CFG) & Top-Down Parsing	Menggunakan BNF dan fungsi eat()
Semantic	Attribute Grammar & Type Checking	Memeriksa konsistensi variabel di AST
TAC	Intermediate Representation (IR)	Mengubah AST menjadi instruksi 3-alamat

Penjelasan Detail:

1. Lexical Analysis → Regular Language

Token-token dalam bahasa pemrograman membentuk *regular language* yang bisa dikenali oleh Finite Automata. Implementasi menggunakan regex yang secara konsep diubah menjadi DFA.

2. Syntax Analysis → Context-Free Grammar

Struktur program dijelaskan dengan CFG dalam bentuk BNF. Parser membaca token sesuai aturan tersebut (top-down parsing).

3. Semantic Analysis → Attribute Grammar

Attribute grammar menambahkan informasi semantik ke node AST. Implementasi menambahkan pengecekan konsistensi variabel.

4. TAC → Intermediate Representation

TAC adalah IR yang *target-independent* dan memudahkan optimasi. TAC yang dihasilkan bisa di-generate ke berbagai arsitektur.

BAB III

KESIMPULAN

3.1 Kesimpulan

Program yang dikembangkan telah berhasil mensimulasikan empat tahapan utama dalam proses kompilasi pada konstruksi *while loop*. Tahapan tersebut meliputi *Lexical Analysis* yang berfungsi memecah *source code* menjadi token-token, *Syntax Analysis* untuk memverifikasi struktur program sekaligus membangun *Abstract Syntax Tree* (AST), *Semantic Analysis* yang memeriksa konsistensi logika program, serta *Three-Address Code* (TAC) Generation yang menghasilkan representasi kode menengah sebagai hasil akhir proses kompilasi.

Selain itu, program telah dilengkapi dengan mekanisme *error handling* yang mampu mendeteksi berbagai jenis kesalahan pada kode sumber. Kesalahan yang dapat diidentifikasi meliputi *syntax error*, yaitu kesalahan pada struktur atau tata bahasa program, serta *semantic error*, yaitu kesalahan logika yang menyebabkan program tidak sesuai dengan aturan semantik meskipun struktur sintaksnya benar.

Secara keseluruhan, implementasi program telah berhasil menerapkan konsep-konsep dasar teori kompilasi yang meliputi penggunaan *Regular Expression* pada proses *lexical analysis*, *Backus-Naur Form* (BNF) atau *Context-Free Grammar* (CFG) pada *syntax analysis*, *attribute grammar* pada *semantic analysis*, serta *intermediate representation* dalam bentuk *Three-Address Code* (TAC). Dengan demikian, program ini dapat menjadi media pembelajaran yang efektif untuk memahami tahapan-tahapan utama dalam proses kompilasi.

REFERENSI

<https://socs.binus.ac.id/2019/12/23/teknik-kompilasi-first-set-pada-top-down-parsing/>

<https://wimpy.blog.binusian.org/2014/03/17/tekkom01/>

<https://www.idn.id/cara-kerja-compiler-dari-kode-sumber-ke-eksekusi-program/>

<https://unstop.com/blog/phases-of-a-compiler>